# RAG-Based Customer Support Assistant using LangGraph and HITL

Low-Level Design Document

Version 1.0

Name : Eesha Haldavnekar

Intern ID : IN226120202

Made with GAMMA

# 1. Module-Level Design

Create a structured breakdown with subsections for each module:

## 1.1 Document Processing Module (PyPDFLoader)

- Responsibility: Load and extract text from PDF documents
- Input: File path to PDF
- Output: Raw text content with metadata (filename, page count)
- Error handling: File not found, corrupted PDF, unsupported format
- Dependencies: PyPDF2, pdfplumber

## 1.2 Chunking Module (RecursiveCharacterTextSplitter)

- Responsibility: Split documents into manageable chunks
- Configuration: chunk_size=1000, chunk_overlap=200
- Strategy: Recursive splitting by separators [\n\n, \n, " ", ""]
- Output: List of text chunks with source metadata
- Rationale: Optimal token length for embeddings and context window

## 1.3 Embedding Module (MiniLM Model)

- Responsibility: Convert text chunks to vector embeddings
- Model: sentence-transformers/all-MiniLM-L6-v2
- Embedding dimension: 384
- Batch processing: Process chunks in batches of 32
- Output: Vector embeddings with chunk references

## 1.4 Vector Storage (ChromaDB)

- Responsibility: Store and index embeddings for fast retrieval
- Schema: {chunk_id, text, embedding, metadata}
- Indexing: HNSW (Hierarchical Navigable Small World)
- Persistence: Local SQLite backend
- Query interface: Similarity search with top-k retrieval

## 1.5 Retrieval Module

- Responsibility: Fetch relevant documents based on query
- Method: Semantic similarity search
- Top-k: Retrieve top 5 documents by default
- Similarity threshold: 0.5 (configurable)
- Output: Ranked list of relevant chunks with scores

## 1.6 Query Processing Module

- Responsibility: Preprocess user queries
- Operations: Lowercasing, whitespace normalization, special character handling
- Validation: Check query length (min 3 chars, max 500 chars)
- Output: Cleaned query ready for embedding

Made with GAMMA

# 2. Data Structures

This section defines the core data structures used throughout the RAG-based customer support assistant system, detailing the schema and purpose of each object for document handling, chunking, embeddings, query-response, and graph state management.

## 2.1 Document Object

The `Document` class represents a PDF document ingested into the system, encapsulating its content and essential metadata.

```
class Document:
    id: str                 # Unique document identifier
    filename: str           # Original PDF filename
    content: str            # Full extracted text
    page_count: int         # Total pages in PDF
    upload_timestamp: datetime # When document was added
    metadata: Dict[str, Any]   # Custom metadata (source, category, etc.)
    chunks: List[Chunk]        # Associated chunks
```

## 2.2 Chunk Structure

The `Chunk` class defines how larger documents are segmented into smaller, manageable pieces for processing and embedding.

```
class Chunk:
    id: str                 # Unique chunk identifier (doc_id:chunk_num)
    document_id: str        # Reference to parent document
    text: str               # Chunk content (max 1000 chars)
    start_char: int         # Character position in original document
    end_char: int           # Character position in original document
    embedding: List[float]     # 384-dimensional vector
    metadata: Dict[str, Any]   # {page_num, section, chunk_index}
```

## 2.3 Embedding Vector

The `EmbeddingVector` class stores the numerical representation of a text chunk, crucial for semantic similarity searches.

```
class EmbeddingVector:
    chunk_id: str           # Reference to chunk
    vector: np.ndarray      # Shape (384,) - MiniLM output
    norm: float             # L2 norm for similarity calculation
    created_at: datetime    # When embedding was generated
```

## 2.4 Query-Response Schema

This schema outlines the structure for incoming user queries (`QueryRequest`) and the system's generated replies (`QueryResponse`).

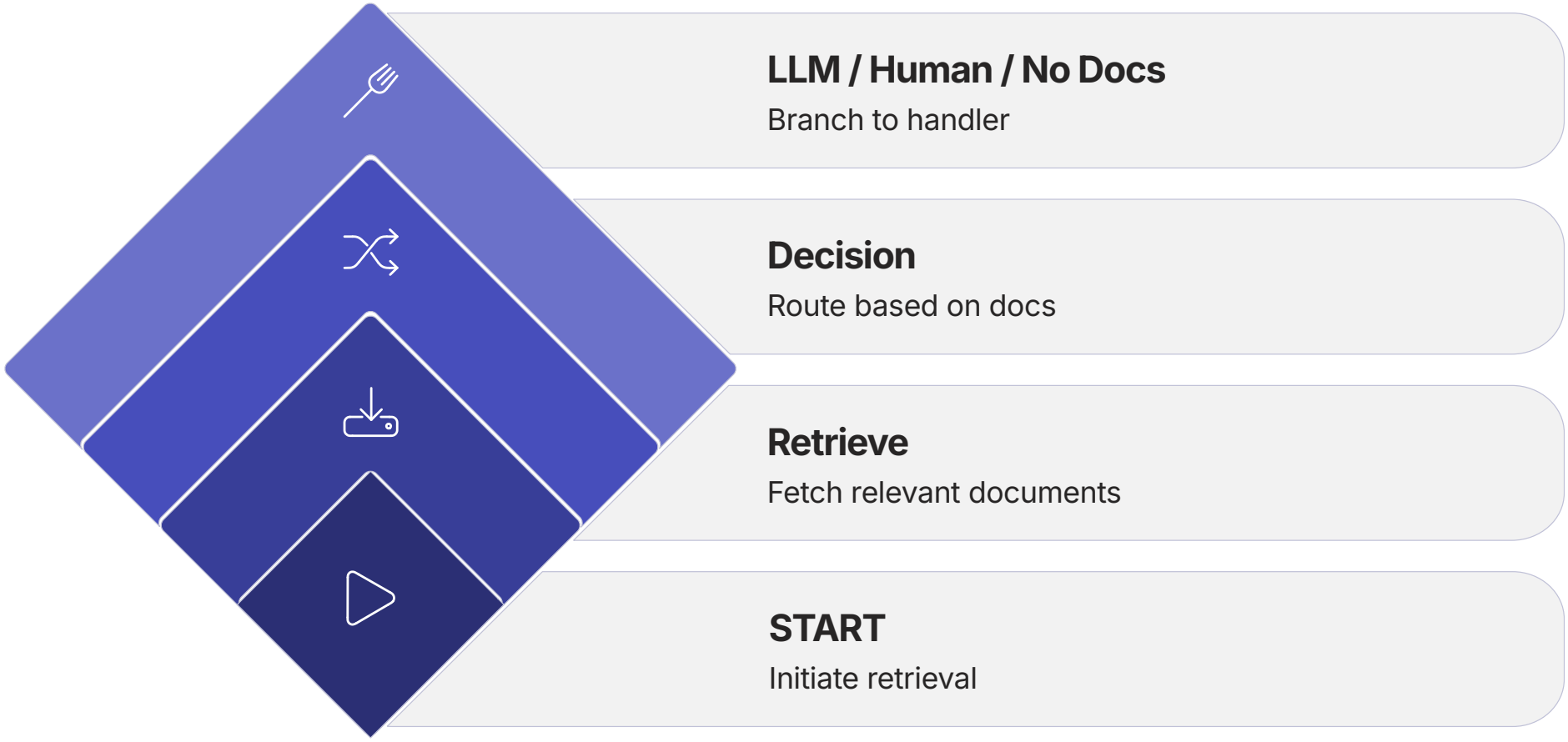
```
class QueryRequest:
    query: str              # User input (3-500 chars)
    user_id: str            # User identifier
    session_id: str         # Session tracking
    timestamp: datetime     # Query submission time
    metadata: Dict[str, Any]   # Additional context

class QueryResponse:
    query_id: str           # Unique query identifier
    answer: str             # Generated response
    source_chunks: List[str]   # IDs of retrieved chunks
    confidence_score: float    # 0.0-1.0 confidence metric
    route: str              # "llm" | "human" | "no_docs"
```

Made with GAMMA

# 3. Workflow Design (LangGraph)

## 3.1 Graph Architecture Overview

The LangGraph workflow implements a state machine with four primary nodes and conditional edges for intelligent routing.




This diagram illustrates the high-level architecture of the LangGraph workflow, showcasing the sequential and conditional execution paths.

## 3.2 Node Definitions

### Node 1: retrieve

- Purpose: Retrieve relevant documents from vector store
- Input: processed_query from GraphState
- Processing:

Made with GAMMA

# 4. Conditional Routing Logic

The routing logic determines whether a query should be handled by the LLM, escalated to a human, or marked as having no relevant documents.

## 4.2 Keyword-Based Detection

**Escalation Keywords (Trigger Human Routing):**

- Legal terms: "legal", "law", "lawsuit", "compliance", "regulation", "GDPR", "CCPA", "contract"
- Enterprise/Critical: "enterprise", "urgent", "critical", "SLA", "breach", "security incident"
- Sensitive: "personal data", "health", "medical", "financial", "account closure"
- Complaint indicators: "complaint", "refund", "cancel", "dispute", "escalate"

**Detection Method:**

```
def detect_escalation_keywords(query: str) -> List[str]:
    query_lower = query.lower()
    detected = []
    for keyword in ESCALATION_KEYWORDS:
        if keyword in query_lower:
            detected.append(keyword)
    return detected
```

## 4.3 Confidence Score Calculation

Confidence is computed as a weighted combination of retrieval quality and query-document relevance:

```
confidence = (avg_retrieval_score × 0.6) + (relevance_score × 0.4)
```

where:

- - avg_retrieval_score = mean of top-k similarity scores (0.0-1.0)
- - relevance_score = semantic similarity between query and retrieved docs (0.0-1.0)

**Thresholds:**

- - confidence ≥ 0.7: High confidence → route to LLM
- - 0.3 ≤ confidence < 0.7: Medium confidence → route to LLM with caution
- - confidence < 0.3: Low confidence → route to human

## 4.4 No Documents Found Scenario

**Trigger Conditions:**

1. ChromaDB returns 0 results
2. All retrieval scores < 0.3 (similarity threshold)
3. Retrieved chunks are too short or fragmented

**Handling:**

```
if len(retrieved_docs) == 0 or max(retrieval_scores) < 0.3:
    route = "no_docs"
    answer = "I don't have information about this topic in my knowledge base.
            Please contact our support team for assistance."
    escalation_reason = "No relevant documents found"
```

## 4.5 Low Confidence Cases

**Trigger Conditions:**

1. Confidence score < 0.3
2. Retrieval scores are borderline (0.3-0.5)
3. Query is ambiguous or multi-part

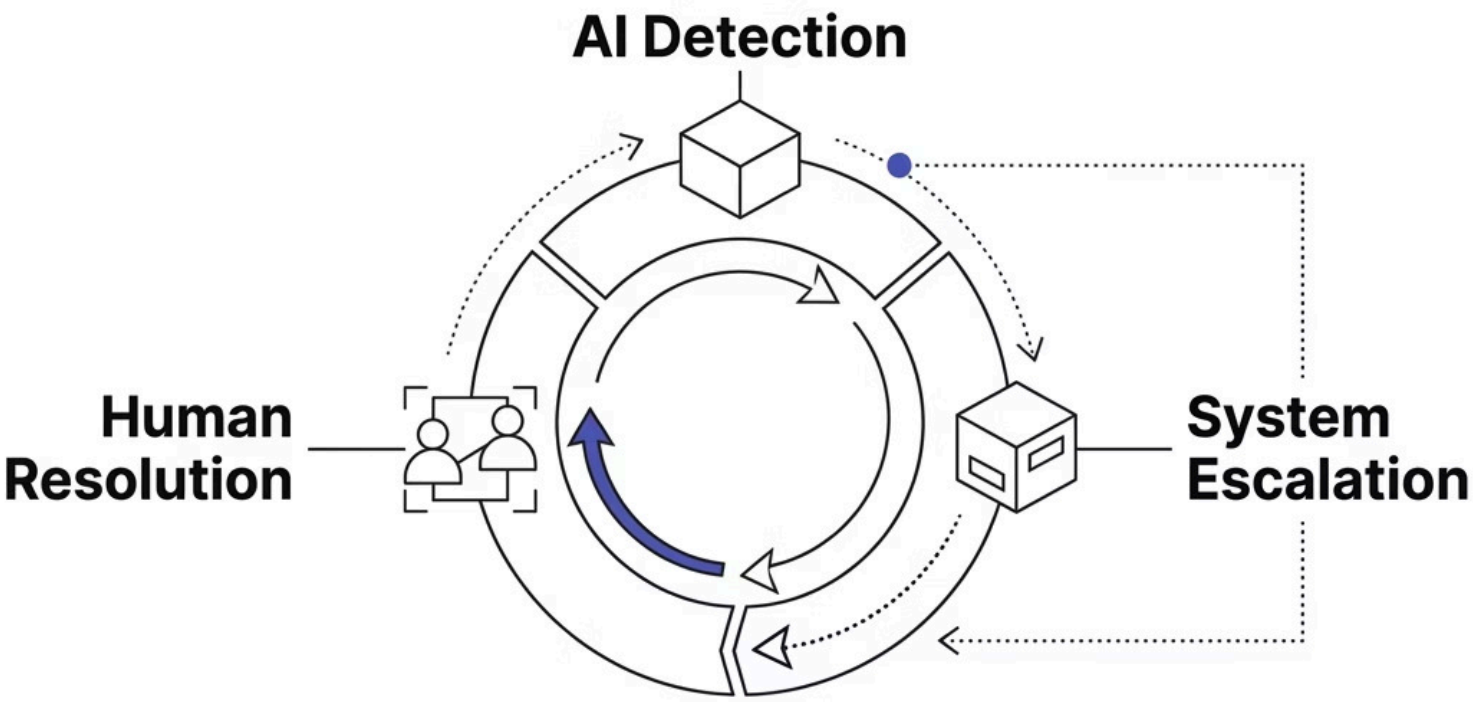
**Handling:**

Made with GAMMA

# 5. HITL (Human-in-the-Loop) Design

## 5.1 HITL Architecture Overview

The HITL module manages the escalation workflow, ensuring seamless handoff from automated systems to human agents and integration of human responses back into the system.




Automated systems escalate unresolved or high-risk tickets.
Human responses are captured to retrain AI and prevent recurrence.
This continuous loop improves detection accuracy.

## 5.2 Escalation Trigger Conditions

Queries are escalated to humans when:

| 1 | 2 |
|---|---|
| **Confidence-Based Escalation**<br>• Confidence score < 0.3<br>• Retrieval results are ambiguous or conflicting<br>• Multiple interpretations of the query | **Keyword-Based Escalation**<br>• Legal/compliance queries<br>• Enterprise/critical issues<br>• Sensitive personal or financial information<br>• Complaint or dispute indicators |

Made with GAMMA

# 6. API / Interface Design

This section outlines the design of the API, Command Line Interface (CLI), webhook integration, authentication mechanisms, and error handling for the RAG Support Assistant system.

## 6.1 REST API Endpoints

**Base URL:** `https://api.support-assistant.com/v1`

### 6.1.1 Query Submission Endpoint

```
POST /queries
Content-Type: application/json

Request Body:
{
  "query": "How do I reset my password?",
  "user_id": "user_12345",
  "session_id": "sess_abcdef",
  "metadata": {
    "source": "web_chat",
    "user_tier": "premium",
    "language": "en"
  }
}

Response (200 OK):
{
  "query_id": "q_xyz789",
  "status": "processing",
  "estimated_wait_time_ms": 2000
}
```

### 6.1.2 Query Result Endpoint

```
GET /queries/{query_id}
Authorization: Bearer {token}

Response (200 OK):
{
  "query_id": "q_xyz789",
  "query": "How do I reset my password?",
  "answer": "To reset your password, follow these steps...",
  "source_chunks": ["chunk_001", "chunk_002"],
  "confidence_score": 0.85,
  "route": "llm",
  "escalated": false,
  "execution_time_ms": 1850,
  "timestamp": "2026-04-25T10:30:00Z"
}

Response (202 Accepted - Still Processing):
{
  "query_id": "q_xyz789",
  "status": "processing",
  "estimated_wait_time_ms": 1500
}
```

### 6.1.3 Escalation Status Endpoint

```
GET /escalations/{ticket_id}
Authorization: Bearer {token}

Response (200 OK):
```

Made with GAMMA

# 7. Error Handling & Recovery

## 7.1 Error Categories and Handling Strategies

### 7.1.1 Document Processing Errors

**Missing or Corrupted PDF**

```
Error Type: FileNotFoundError, PDFException
Trigger: PDF file not found or corrupted during loading
Handling:
1. Log error with file path and timestamp
2. Return user-friendly message: "Unable to load document. Please check file format."
3. Retry mechanism: Attempt 3 times with exponential backoff (1s, 2s, 4s)
4. Fallback: Skip document and continue with other documents
5. Alert: Notify admin if critical document fails

Code:
try:
    document = PyPDFLoader(file_path).load()
except FileNotFoundError:
    logger.error(f"PDF not found: {file_path}")
    raise DocumentProcessingError("File not found")
except PDFException as e:
    logger.error(f"PDF corrupted: {file_path}, Error: {str(e)}")
    raise DocumentProcessingError("PDF corrupted or unsupported format")
```

### 7.1.2 Chunking Errors

**Empty Document**

```
Error Type: ValueError
Trigger: Document has no content after extraction
Handling:
1. Check document length before chunking
2. If empty, log warning and skip
3. Return empty chunk list
4. Continue processing other documents

Code:
if len(document.content) == 0:
    logger.warning(f"Empty document: {document.id}")
    return []
```

**Chunk Size Validation**

```
Error Type: ValueError
Trigger: Chunk size configuration invalid
Handling:
1. Validate chunk_size > 0 and chunk_overlap < chunk_size
2. Set defaults if invalid: chunk_size=1000, chunk_overlap=200
3. Log warning about adjustment
4. Continue with corrected values

Code:
if chunk_size <= 0 or chunk_overlap >= chunk_size:
    logger.warning(f"Invalid chunk config. Using defaults.")
    chunk_size, chunk_overlap = 1000, 200
```

### 7.1.3 Embedding Generation Errors

**Model Loading Failure**

Made with GAMMA

```
Error Type: RuntimeError, ModelNotFoundError
```

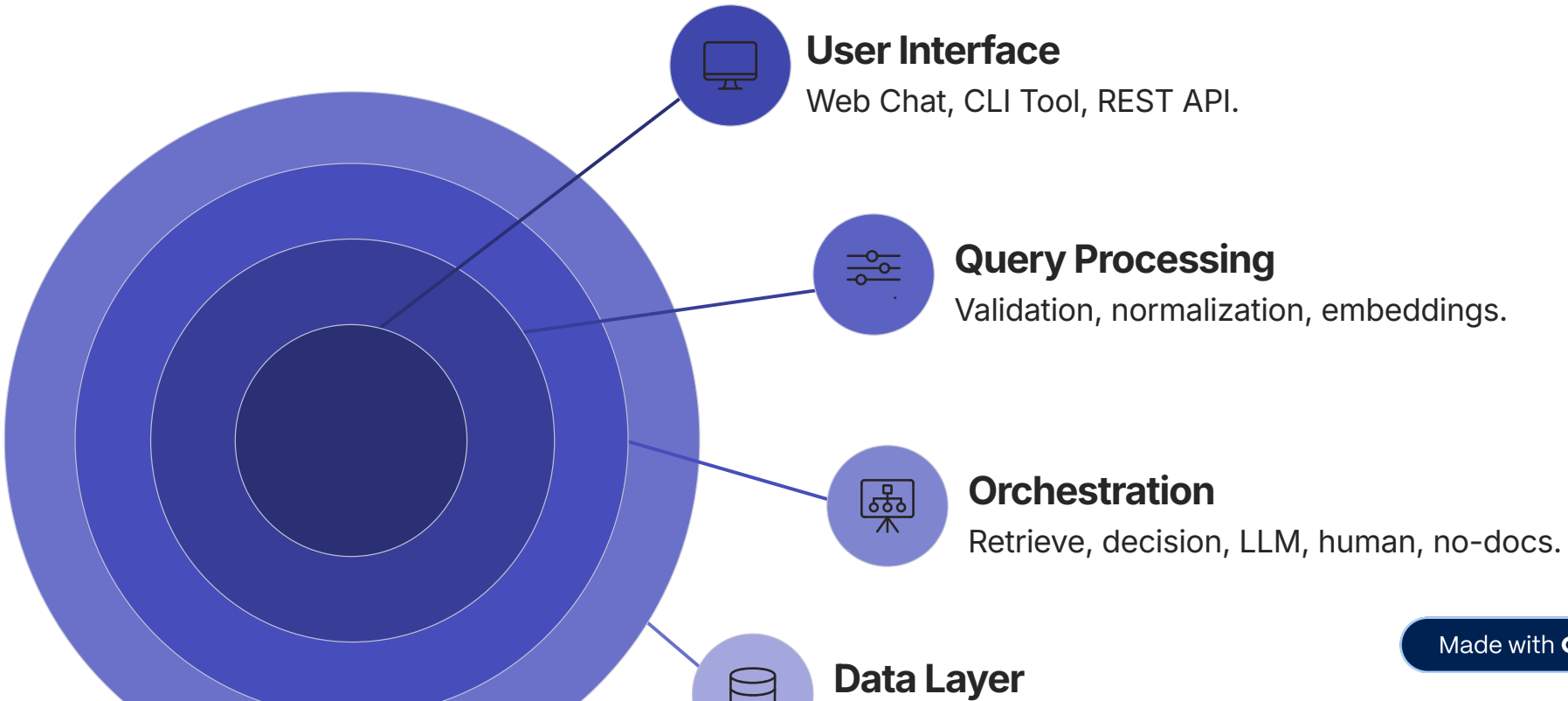
# 8. Implementation Summary & Architecture Diagram

## 8.1 Technology Stack

| Component | Technology | Version | Purpose |
|---|---|---|---|
| Document Loading | PyPDFLoader (LangChain) | 0.1.0+ | Extract text from PDFs |
| Text Splitting | RecursiveCharacterTextSplitter | LangChain | Chunk documents intelligently |
| Embeddings | sentence-transformers (MiniLM) | 2.2.0+ | Generate 384-dim vectors |
| Vector Store | ChromaDB | 0.4.0+ | Store and retrieve embeddings |
| LLM | OpenAI GPT-4 / Claude | Latest | Generate responses |
| Orchestration | LangGraph | 0.1.0+ | State machine workflow |
| API Framework | FastAPI | 0.100.0+ | REST API endpoints |
| Database | PostgreSQL | 14+ | Store escalations, feedback |
| Cache | Redis | 7.0+ | Cache embeddings, responses |
| Logging | Python logging + JSON | Standard | Structured logging |
| Monitoring | Prometheus + Grafana | Latest | Metrics and dashboards |

## 8.2 System Architecture Overview

The system architecture is organized into distinct layers, ensuring modularity, scalability, and maintainability. The User Interface Layer interacts with the Query Processing Layer, which then feeds into the LangGraph Orchestration for intelligent decision-making before accessing the Data Layer.




Made with GAMMA